CSE 4304-Data Structures Lab. Winter 2022
Date: Sept 20, 2022.
Target Group: All
Topic: Binary Tree

Instructions:

- Task naming format: fullID_T01L05_2A.c/CPP
- If you find any issues in problem description/test cases, comment in the google classroom.
- If you find any tricky test case that I didn't include but others might forget to handle, please comment! I'll be happy to add.
- Use appropriate comments in your code. This will help you to recall the solution in the future easily.
- Obtained marks will vary based on the efficiency of the solution.
- Do not use <bits/stdc++.h> library, add the necessary libraries one after another.
- Do not use any built-in functions or any STL libraries. If the solution is extremely difficult without the usage of such components, discuss it with the instructors.
- Modified sections will be marked with BLUE color.

Group	**Tasks**
1B	1 2 3 4
2B	1 2 3 5
2A	1 2 5 6
1A	1 2 5 6

Task 1

Implement the basic operations of Binary tree. Your program should include the following functions:

1. **Insert**:
   - Assuming each node contains a unique value.
   - Input starts with a number N (representing the number of nodes), followed by N lines containing the info of node.
   - Every line contains three values 'data', 'parent' and '1(left child) or 2(right child)'. For the root, the second and third parameter is Null (0).
   - After successful insertion, print the entire tree using preorder traversal. Beside each node, print the info of it's parent as well.
2. **Removal**: if an internal node is removed, its subtrees are also removed.
3. **Search**: Returns the node if it is present and prints it's description.
4. **Height**: Returns the height of the tree, where height of a leaf node is 0.

Input	**Output**
7 1 0 0 2 1 1 3 1 2 5 2 2 4 2 1 6 3 1 7 3 2	1(N) 2(1) 4(2) 5(2) 3(1) 6(3) 7(3) (preorder) **Note**: The tree looks like as follows: 1 / \ 2 3 / \ / \ 4 5 6 7
3 3 (search 3)	Present, Left(6), Right(7)
2 6 (remove 6)	1(N) 2(1) 4(2) 5(2) 3(1) 7(3) (preorder)
3 1 (search 1)	Present, Left(2), Right(3)
3 5	Present, Left(null), Right(null)
3 6	Not present
3 3	Present, Left(null), Right(7)
4	2
2 3	1(N) 2(1) 4(2) 5(2) (preorder)
4	2
3 3	Not present
2 2	1(N) (preorder)
4	0

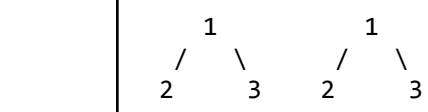
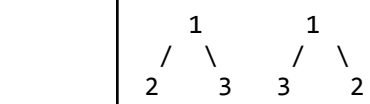
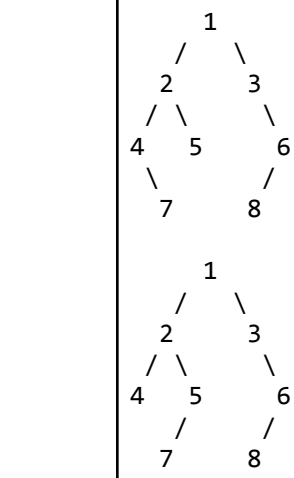
Task 2

For a binary tree, show the result of level-order, inorder, preorder and postorder traversal. Input policy is same as Task-1.

Input	Output	Commnet
5 A B A 1 C A 2 D B 1 E B 2	A B C D E (level order) D B E A C (Inorder) A B D E C (Preorder) D E B C A (Postorder)	Original Tree A / \ B C / \ D E
9 1 0 0 2 1 1 3 1 2 5 2 2 6 3 1 7 3 2 4 2 1 9 6 2 8 5 1	1 2 3 4 5 6 7 8 9 (level order) 4 2 8 5 1 6 9 3 7 (Inorder) 1 2 4 5 8 3 6 9 7 (Preorder) 4 8 5 2 9 6 7 3 1 (Postoder)	Original tree: 1 / \ 2 3 / \ / \ 4 5 6 7 / \ 8 9

Task 3

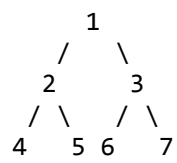
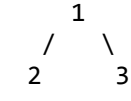
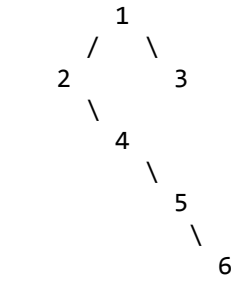
Given two binary trees, the task is to find if both of them are identical or not.

Input	**Output**
	Yes
	No
	No

Task 4

The task is to print the top view of the binary tree. Top view of a binary tree is the set of nodes visible when the tree is viewed from the top.

Note: Return nodes from leftmost node to rightmost node.

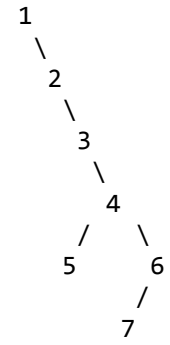
Input	**Output**
	4 2 1 3 7
	2 1 3
	2 1 3 6

Hint: The idea is to do something similar to vertical Order Traversal. Like vertical Order Traversal, we need to put nodes of the same horizontal distance together. We do a level order traversal so that the topmost node at a horizontal node is visited before any other node of the same horizontal distance below it.

Task 5

Given a binary tree with N nodes, your task is to print all the paths from the root to the leaves along with the path length. You can print the paths in any order. Make sure the path starts from the root and ends at a leaf. Follow the input policy as Task 1.

Input	Output	Comment
7 1 0 0 3 1 2 2 1 1 4 2 1 5 2 2 7 3 2 6 3 1	1 2 4 (3) 1 2 5 (3) 1 3 6 (3) 1 3 7 (3)	Original tree: 1 / \ 2 3 / \ / \ 4 5 6 7
6 1 0 0 2 1 1 3 1 2 4 2 2 5 4 2 6 5 2	1 2 4 5 6 (5) 1 3 (2)	Original tree: 1 / \ 2 3 \ 4 \ 5 \ 6
8 1 0 0 2 1 1 3 1 2 8 3 2 4 2 2 7 3 1 5 4 2 6 5 2	1 2 4 5 6 (5) 1 3 7 (3) 1 3 8 (3)	Original tree: 1 / \ 2 3 \ / \ 4 7 8 \ 5 \ 6
9 1 0 0 2 1 1 3 1 2 5 2 2 6 3 1 7 3 2 4 2 1 9 6 2 8 5 1	1 2 4 (3) 1 2 5 8 (4) 1 3 6 9 (4) 1 3 7 (3)	Original tree: 1 / \ 2 3 / \ / \ 4 5 6 7 / \ 8 9

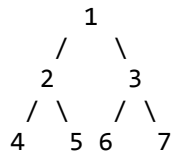
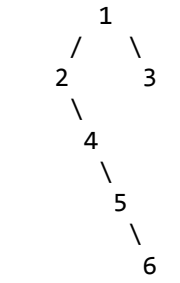
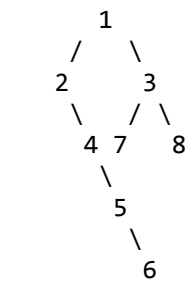
7 1 0 0 2 1 2 3 2 2 4 3 2 5 4 1 6 4 2 7 6 1	1 2 3 4 5 (5) 1 2 3 4 6 7 (6)	

Hint: Use any traversal method. May use parent pointers.

Task 6

Given two nodes in a binary tree, your task is to find out if they belong to the same level or not. If so, print their levels as well. Assume that the root is at level 0.

- At first take the input for N nodes of the tree as mentioned in Task-1.
- After that take an input ‘Q’ representing the number of queries. The next ‘Q’ lines contain two values representing the data in the two nodes.

Input	**Output**	**Comment**
7 1 0 0 3 1 2 2 1 1 4 2 1 5 2 2 7 3 2 6 3 1 4 1 3 2 3 4 6 4 3	 No Yes (Level 1) Yes (Level 2) No	
6 1 0 0 2 1 1 3 1 2 4 2 2 5 4 2 6 5 2 3 1 5 2 4 2 3	 No No Yes (Level 1)	
8 1 0 0 2 1 1 3 1 2 8 3 2 4 2 2 7 3 1 5 4 2 6 5 2 4 4 7 5 6 2 3 5 8	 Yes (Level 2) No Yes (Level 1) No	

9 1 0 0 2 1 1 3 1 2 5 2 2 6 3 1 7 3 2 4 2 1 9 6 2 8 5 1 8 8 9 4 7 2 6 2 3 5 6 6 5 5 4 5 3	 Yes (Level 3) Yes (Level 2) No Yes (Level 1) Yes (Level 2) Yes (Level 2) Yes (Level 2) No	